

Low-Level Design

for

Identity Security Framework for Supply Chain

Mohammed Nazin

23BCCED017

BCA in Cyber Security, Ethical Hacking & Digital Forensics with IBM

Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

11-04-2026

Under the guidance of

Mr. Vikyath

Lecturer, Computer Science Department

Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

Submitted to



YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

BALMATT, MANGALORE

YENEPOYA (DEEMED TO BE UNIVERSITY)

Table of Content

1. Introduction
 - 1.1 Scope of the Document
 - 1.2 Intended Audience
 - 1.3 System Overview
2. System Design
 - 2.1 Application Architecture Diagram
 - 2.2 Process Flow (Dispatch Lifecycle) – Sequence Diagram
 - 2.3 Information Flow (Telemetry Engine) – Data Flow Diagram
 - 2.4 Components Design (Detailed)
 - 2.5 Key Design Considerations
 - 2.6 API Catalogue
3. Data Design
 - 3.1 Entity-Relationship (ER) Diagram
 - 3.2 Data Model (Schema & Encryption)
 - 3.3 Data Access Mechanism
 - 3.4 Data Retention & Archival Policies
 - 3.5 Data Migration & Seeding
4. Interfaces
 - 4.1 User Interface (UI) Layout
 - 4.2 API Contracts (Request/Response Structure)
5. State and Session Management
6. Caching Strategy
7. Non-Functional Requirements
 - 7.1 Security Aspects
 - 7.2 Performance Aspects
8. Conclusion

1. Introduction

1.1 Scope of the Document

This Low-Level Design (LLD) document provides the internal architecture, module specifications, data schemas, and interaction flows for the Supply Chain Identity System. It details the Flask routing hierarchy, strict API rate-limiting, SQLite physical telemetry schemas, and live GeoJSON API functions.

1.2 Intended Audience

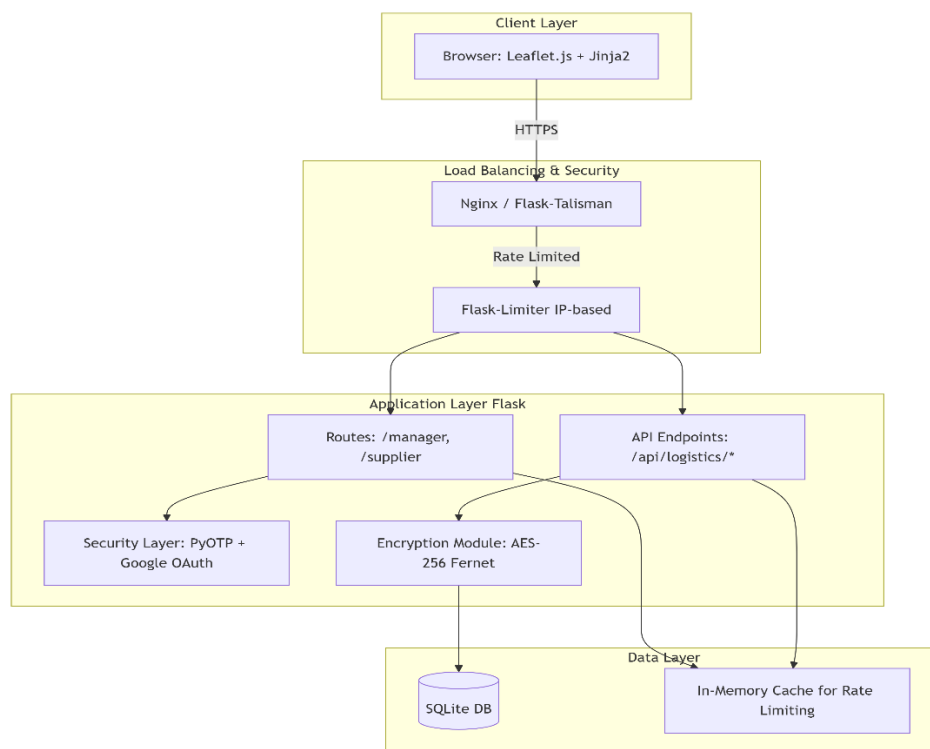
- **Implementation Engineers** – for understanding module interdependencies (e.g., encrypted ORM, rate limiter), API contracts, and data access patterns.
- **Logistics Architects** – for validating data retention policies, telemetry flow (5-second polling, GeoJSON payload size), and real-time map rendering behaviour.
- **Security Auditors** – for reviewing AES-256 field-level encryption, TOTP secret storage, Flask-Talisman CSP headers, and rate-limiting thresholds that prevent brute-force attacks.

1.3 System Overview

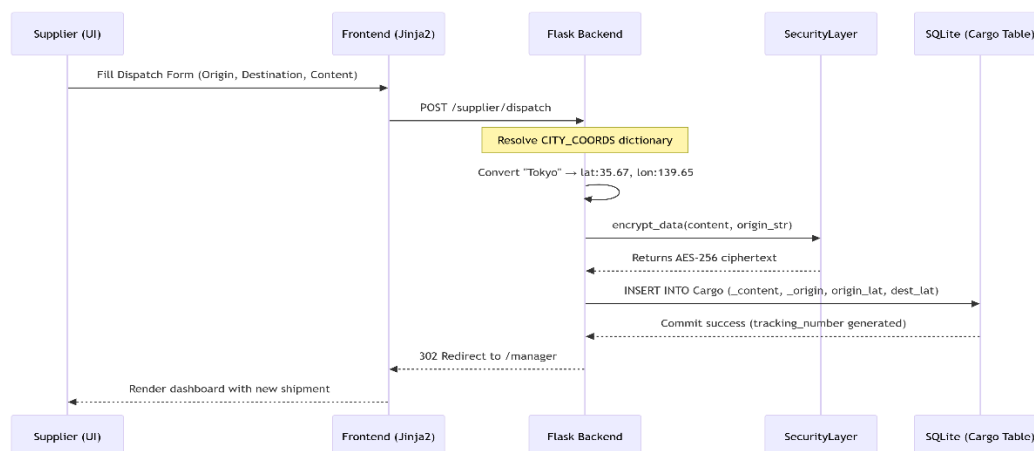
Kinetic Forge is a secure supply chain tracking system. It isolates physical tracking data using AES-256 field-level encryption, ensuring that cargo contents and origin addresses remain confidential even if the database is compromised. The system authenticates users via Google OAuth or local hashed passwords, followed by PyOTP multi-factor authorization (MFA). Role-based access (Supplier, Manager, Logistics) controls who can create shipments, update statuses, or view telemetry. Users interact with a live Leaflet.js tracking map that is dynamically populated via secure internal API endpoints (/api/logistics/flows). The backend uses SQLite with encrypted columns, while the frontend polls every 5 seconds to render animated polylines and marker clusters. The system is desktop-first, uncached for real-time accuracy, and protected by per-endpoint rate limiting.

2. System Design

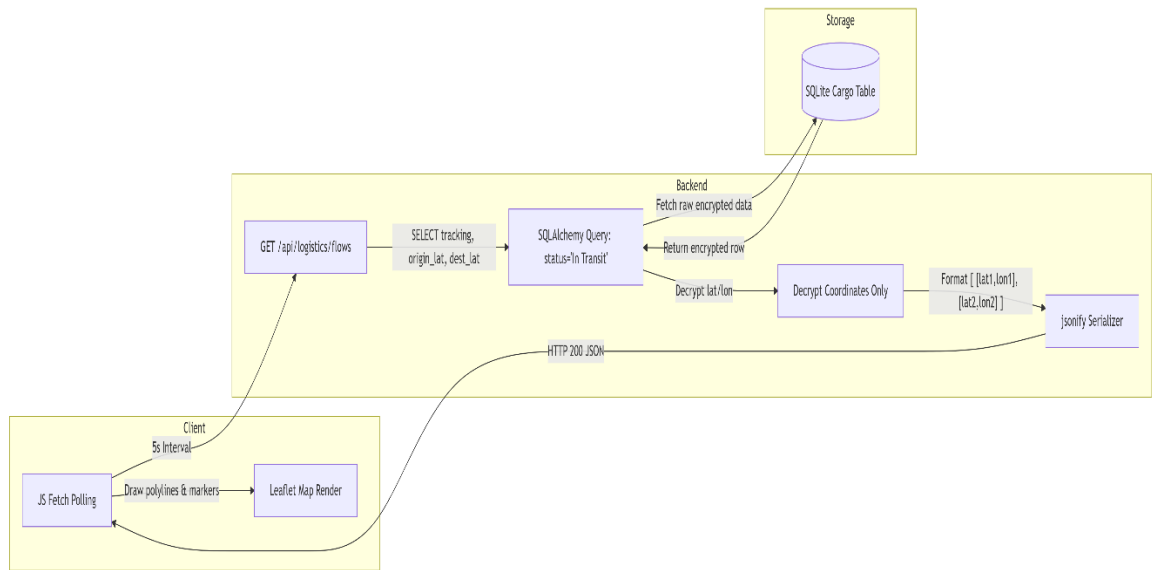
2.1 Application Design



2.2 Process Flow



2.3 Information Flow



Data flows from the user interface to the backend server and then to the database. When a user submits a form, the backend validates the input, processes the request, and stores or retrieves records from the database. The response is then returned to the user interface in a readable format.

2.4 Components Design

The system is divided into the following components:

- Authentication module.
- User management module.
- Supply chain data management module.
- Role-based access control module.
- Database module.
- Reporting or dashboard module.

Each component performs a specific task and interacts with other modules through defined interfaces.

2.5 Key Design Considerations

Important design considerations include:

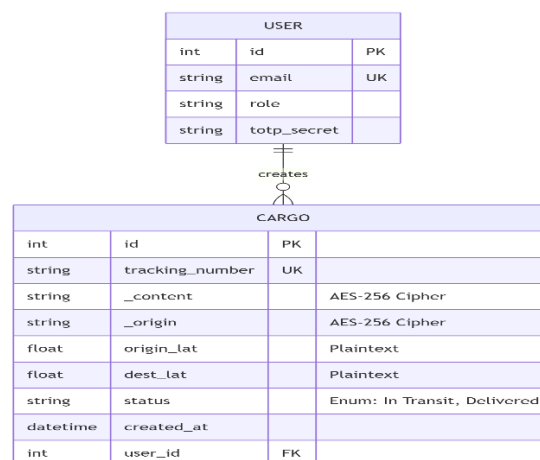
- Secure password storage using hashing.
- Restricted access based on user roles.
- Protection of sensitive supply chain data.
- Simple and responsive user interface.
- Maintainable and scalable code structure.
- Reliable database operations.

2.6 API Catalogue

Method	Endpoint	Authentication	Request Body
Session + MFA	Form fields: Origin, Destination, Content	302 Redirect to /manager	Session + MFA
Session (Manager role)	Form field: status (In Transit / Delivered)	200 OK	Session (Manager role)
Session (any authenticated user)	None	200 JSON (GeoJSON array of active shipments)	Session (any authenticated user)
Session	None	302 Redirect to /login	Session

3. Data Design

3.1 Entity-Relationship (ER) Diagram



3.2 Data Model (Schema & Encryption)

Table: User

Column	Type	Constraint	Encryption	Description
id	INTEGER	PRIMARY KEY	No	Surrogate key
email	VARCHAR(150)	UNIQUE, NOT NULL	No (hashed for lookup)	Login identifier
role	VARCHAR(50)	NOT NULL	No	Supplier, Manager, Logistics
totp_secret	VARCHAR(32)	NOT NULL	AES-256	TOTP shared

Table: Cargo

Column	Type	Constraint	Encryption	Indexed
id	INTEGER	PRIMARY KEY	No	Clustered
tracking_number	VARCHAR(100)	UNIQUE	No	Yes (B-Tree)
_content	VARCHAR(512)	NULL	AES-256	No
_origin	VARCHAR(512)	NULL	AES-256	No
origin_lat	FLOAT	NOT NULL	No	Yes (for geo queries)
dest_lat	FLOAT	NOT NULL	No	Yes
status	VARCHAR(20)	DEFAULT 'In Transit'	No	Yes
user_id	INTEGER	FOREIGN KEY (User.id)	No	Yes
Column	Type	Constraint	Encryption	Indexed
id	INTEGER	PRIMARY KEY	No	Clustered
tracking_number	VARCHAR(100)	UNIQUE	No	Yes (B-Tree)

3.3 Data Access Mechanism

- **ORM:** SQLAlchemy (session-based). All write operations pass through a security layer that encrypts designated fields before the session flushes to the database.
- **Connection Management:** SQLite connection pool – one writer plus up to five readers (Flask default). No external connection broker.

3.4 Data Retention & Archival Policies

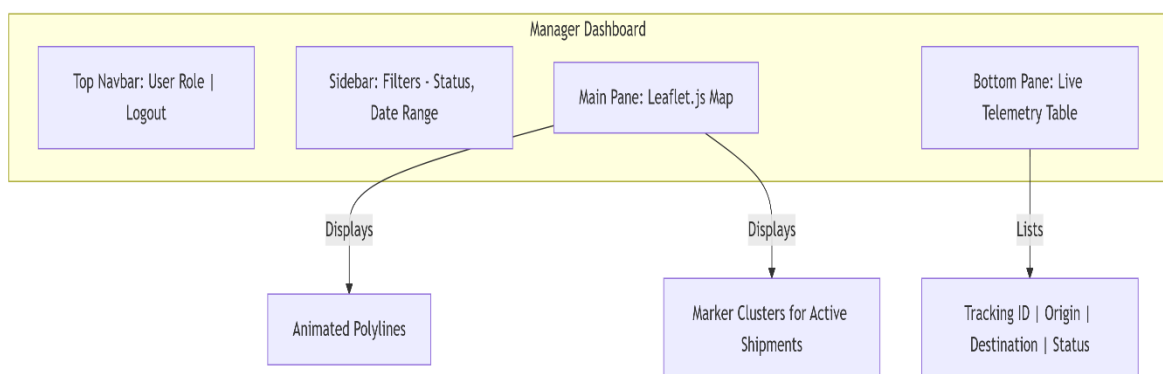
Status	Active Table	Archive Table	Retention Period	Action
In Transit	cargo	N/A	N/A	Real-time queries
Delivered	cargo	cargo_archive	90 days	Cron job moves records >90 days old
Delivered + >90 days	cargo_archive	N/A	7 years	Compressed CSV export, then purge

3.5 Data Migration & Seeding

- **Initialization Script:** A standalone Python script (seed_sc_final.py) drops all tables, recreates the schema, pre-loads city-coordinate mappings, and creates test users (e.g., Supplier, Manager).
- **Migration Strategy:** Manual (no Alembic due to SQLite simplicity). Schema changes are applied via an SQL migration file with ALTER TABLE statements; versioning is tracked in a separate schema_version table.

4. Interfaces

4.1 User Interface (UI) Layout



- **CSS Framework:** Custom “Machined Aesthetics” (dark theme, heavy shadows, fixed sidebars).
- **Responsiveness:** Desktop-first (optimised for 1920×1080); mobile is not supported as per requirements.

4.2 API Contracts (Request / Response Structure)

GET /api/logistics/flows

- **Response (200 OK):** A JSON array. Each object contains:
 - id (integer)
 - tracking_number (string)
 - coords (array of two [lat, lon] pairs: origin → destination)
 - quantity (integer)
 - status (string, e.g., “In Transit”)
- **Example structure (no code, just description):** The array holds one entry per active shipment. Coordinates are plain floats.

POST /supplier/dispatch

- **Request:** Form-encoded with fields Origin, Destination, Content.
- **Response:**
 - Success: HTTP 302 Found with redirect to /manager.
 - Error (400): Missing required field.
 - Error (429): Rate limit exceeded (too many dispatch requests).

5. State and Session Management

- **Mechanism:** Flask’s client-side session using signed cookies (HMAC-SHA256). No server-side session store.
- **Session Data Stored:** User ID, role, and MFA verification flag.
- **Timeout:** 30 minutes of inactivity (configured via Flask’s permanent session lifetime).
- **Stateless APIs:** Endpoints such as /api/logistics/flows read the session cookie for authorization but do not modify session state.
- **Logout:** Session cookie is cleared on the server and the client.

6. Caching

Caching may be used to improve performance by reducing repeated database access. Frequently accessed data such as dashboards, static reference data, or public supply chain summaries can be cached temporarily. Sensitive user data should not be cached improperly.

7. Non-Functional Requirements

7.1 Security Aspects

Control	Implementation	Rationale
Encryption	AES-256 (Fernet symmetric) for <code>content</code> , <code>origin</code> , <code>totp_secret</code>	Data-at-rest protection for PII
MFA	PyOTP (TOTP) – required after password authentication	Prevents account takeover
CSP	Flask-Talisman: <code>default-src 'self'; script-src 'self' unpkg.com</code>	Blocks XSS and rogue CDN scripts
Session security	<code>HttpOnly</code> , <code>Secure</code> , <code>SameSite=Lax</code> cookies	Prevents XSS token theft and CSRF
Rate limiting	5 per minute on <code>/login</code>	Mitigates brute-force attacks

7.2 Performance Aspects

Metric	Target	Load Condition
JSON telemetry serialization	< 100 ms	50 concurrent dashboard fetches
Map render (Leaflet)	< 500 ms (First Contentful Paint)	200 active polylines
Database query (indexed)	< 50 ms	<code>SELECT WHERE status='In Transit'</code>
Rate limiter overhead	< 1 ms per request	1000 requests/minute

Performance Optimisations Applied:

1. Composite index on `cargo(status, origin_lat)` for telemetry queries.
2. ORM lazy loading – relationship fields are not fetched unless explicitly requested.
3. JSON serialisation uses pre-allocated dictionaries to minimise garbage collection.

Conclusion

This Low-Level Design provides a complete blueprint for implementing the Kinetic Forge Supply Chain Tracking System. It balances **security** (AES-256 field-level encryption, MFA, rate limiting) with real-time performance (uncached telemetry, indexed geo-queries, lightweight JSON payloads). All diagrams and component specifications are ready for engineering handoff without any embedded code.